

TOBB ETU GRADUATION PROJECT

Team Name: Orion-AI

Project Name: Invisio

Members:

- Mehmet Umur ÖZÜ
- Emre KARADOĞAN
- Kübra ARSLAN
- Fatih BAYAZIT

Low-Level Design Report

Low-Level Design Report Feb 14, 2025

This report is submitted to the TOBB ETU University in partial fulfillment of the requirements of the Graduation Project course BIL496 and YAP496.

Table of Contents

Team Name: Orion-AI	1
Project Name: Invisio	1
Members:	1
Table of Contents	2
1. Introduction	4
1.1. Object design trade-offs	5
1.1.1. Automation vs User Control	5
1.1.2. Accuracy vs. Performance	5
1.1.3. Privacy vs. Personalization	6
1.1.4. Security vs Usability	6
1.1.5. Stateless Architecture vs System Efficiency	7
1.1.6. Model Generalization vs Task-Specific Optimization	7
1.1.7. Feature Richness vs Interface Simplicity	8
1.2. Interface Documentation Guidelines	8
1.2.1. Frontend–Backend Interface	8
1.2.2. Backend–AI Processing Interface	9
1.2.3. Data Format Specifications	10
1.2.4. Error Handling Guidelines	10
1.2.5. Security Interface Constraints	11
1.2.6. External Dependencies Interface	11
1.2.7. Logging and Monitoring Interface	11
1.3. Engineering Standards	12
1.3.1. IEEE Documentation Principles	12
1.3.2. UML Modeling Standards	12
1.3.3. Design and Architectural Principles	14
1.3.4. Coding Standards and Conventions	14
1.3.5. Version Control and Configuration Management	14
1.3.6. Ethical and Data Handling Standards	14
1.4. Definitions, Acronyms, and Abbreviations	15
1.4.1. Definitions	15
1.4.2. Acronyms	16
2. Packages	18
2.1 Server	18
2.1.1 Model	18
2.1.2 Controller	19
2.2 Client	20
2.2.1 View	20
3. Class Interfaces	21

3.1 Server	21
3.1.1 Model	21
3.2 Client	38
3.2.1 View	38
4. Glossary	47
5. References	53
6. Appendix	54
6.1 Appendix A - Figure 1: Model Package Diagram	54
6.2. Appendix B - Figure 2: Controller Package Diagram	54
6.3. Appendix C - Figure 3: View Package Diagram	55

1. Introduction

The Invisio system is an AI-powered, web-based photo editing platform designed to automate complex image enhancement operations through deep learning models. While the High-Level Design (HLD) document defined the architectural structure, subsystem decomposition, and design rationale of the system, this Low-Level Design (LLD) document specifies the detailed implementation-oriented structure required to realize the system.

The purpose of this LLD is to translate architectural abstractions into concrete, implementable software components. It defines the internal structure of the system in terms of packages, classes, interfaces, data flow responsibilities, method signatures, and interaction sequences. The document ensures traceability between HLD components and their corresponding implementation-level representations.

The system implementation follows the **Model–View–Controller (MVC)** architectural pattern. This decision was made to enforce strict separation of concerns, improve maintainability, and ensure testability. In this structure:

- The **Model** layer encapsulates domain entities, AI processing pipelines, validation logic, and memory management.
- The **View** layer is responsible solely for presentation and user interaction.
- The **Controller** layer coordinates requests, validates inputs, invokes appropriate model pipelines, and returns standardized responses.

Adopting MVC enables modular development, simplifies debugging and testing, and prevents tight coupling between user interface components and AI processing logic. Since Invisio integrates computationally intensive AI pipelines, isolating model logic from presentation logic is essential to preserve architectural clarity and long-term extensibility.

Furthermore, the LLD defines standardized interfaces for AI pipelines to ensure consistent behavior across heterogeneous deep learning models. Each transformation (e.g., inpainting, style transfer, beautification, colorization) must comply with a common execution contract. This approach reduces integration complexity and supports future expansion without architectural restructuring.

This document is structured as follows:

- Section 1 defines foundational design considerations and engineering standards.
- Section 2 introduces the MVC-based package structure.
- Section 3 details class interfaces within the Model, Controller, and View layers.
- Section 4 describes interaction flows and sequence behaviors.
- Section 5 explains error handling and edge-case management.
- Section 6 discusses testability and traceability with respect to the HLD.

The objective of this LLD is not to provide source code, but to eliminate technical ambiguity prior to implementation and to establish a robust, traceable, and academically rigorous design blueprint for the Invisio system.

1.1. Object design trade-offs

Designing an AI-powered, privacy-oriented, web-based image editing system requires balancing multiple conflicting objectives. In Invisio, architectural and object-level design decisions were made by carefully evaluating trade-offs between performance, usability, security, privacy, and maintainability.

The following subsections describe the key design trade-offs considered during the low-level design phase.

1.1.1. Automation vs User Control

Invisio follows an AI-first design philosophy. Complex tasks such as segmentation, inpainting, background replacement, and facial enhancement are performed automatically by deep learning models. However, fully automated systems may produce imperfect outputs, especially in edge cases such as:

- Complex object boundaries
- Overlapping foreground elements
- Unusual lighting conditions

Providing full manual control could increase correction accuracy, but it would also:

- Increase interface complexity
- Require technical expertise
- Contradict the goal of simplicity for non-technical users

Therefore, Invisio adopts a hybrid approach. AI models perform the primary editing operations, while user interaction is limited to structured, high-level refinement actions. Pixel-level manual tools are intentionally excluded to maintain clarity and usability. This balance ensures accessibility while still allowing corrective intervention when necessary.

1.1.2. Accuracy vs. Performance

Deep learning models often present a trade-off between accuracy and computational efficiency:

- Larger and more complex models typically produce higher-quality results.
- Higher-quality models require more GPU memory and longer inference times.
- Longer inference times negatively impact user experience in web environments.

Since Invisio is designed as an interactive web application, predictable and bounded latency is prioritized over absolute state-of-the-art accuracy.

To address this trade-off:

- Extremely heavy generative models are avoided as default processing options.
- Resolution scaling strategies may be applied for performance consistency.

The system targets fast and stable responses rather than maximum theoretical accuracy.

1.1.3. Privacy vs. Personalization

Advanced personalization features—such as edit history tracking, user preference modeling, or adaptive fine-tuning—would require persistent storage of user data. However, persistent storage introduces:

- Legal and regulatory risks under GDPR and KVKK
- Increased attack surface
- Long-term data management complexity

Invisio follows a privacy-by-design architecture:

- Images are processed only within a temporary session.
- No images are stored permanently by default.
- Users can interact anonymously without registration.
- No long-term behavioral tracking is implemented.

As a result, personalization capabilities are intentionally limited in favor of strict data minimization and regulatory compliance.

1.1.4. Security vs Usability

High-security mechanisms can sometimes reduce ease of use. Examples include:

- Strict authentication requirements
- Aggressive rate limiting
- Strong input validation constraints
- File size limitations

While these mechanisms increase system protection, they may also introduce user friction.

In Invisio:

- All communication occurs over HTTPS.
- Uploaded files are validated before processing.
- NSFW content filtering is enforced.

- However, mandatory user registration is not required.

This approach ensures security by default while minimizing unnecessary barriers for legitimate users.

1.1.5. Stateless Architecture vs System Efficiency

Invisio is designed as a stateless system where each request is processed independently. Stateless architecture provides several advantages:

- Improved scalability
- Easier horizontal deployment
- Better fault isolation
- Stronger privacy guarantees

However, statelessness also introduces certain inefficiencies:

- No cross-session caching
- Repeated processing for identical inputs
- Potential cold-start overhead

To mitigate these effects:

- Models are loaded at application startup to reduce initialization delay.
- In-memory session handling is used during active processing.

Persistent caching across sessions is intentionally avoided to preserve privacy and architectural clarity.

1.1.6. Model Generalization vs Task-Specific Optimization

A unified multi-purpose model would simplify system structure but may reduce task-specific performance.

On the other hand, using separate models for each editing task increases:

- System modularity
- Processing quality
- Upgrade flexibility

Invisio adopts a task-based modular architecture:

- Each feature (e.g., inpainting, style transfer, colorization) is encapsulated in a dedicated processing module.
- Models are abstracted behind a processing interface.

This design supports future model replacement or improvement without affecting higher

architectural layers.

1.1.7. Feature Richness vs Interface Simplicity

Adding numerous editing options and parameter controls could increase system flexibility but would also:

- Increase cognitive load
- Complicate the user interface
- Reduce accessibility for non-expert users

Invisio deliberately limits feature exposure:

- Advanced parameters are not directly configurable by users.
- Editing actions are presented as high-level tasks.
- Manual editing tools are excluded in the initial version.

This trade-off ensures clarity, ease of use, and consistent user experience.

1.2. Interface Documentation Guidelines

This section defines the interface specifications and communication rules between system components in Invisio. The purpose of these guidelines is to ensure modularity, consistency, and maintainability across all system layers. Invisio follows a layered architecture consisting of:

- Frontend (User Interface Layer)
- Backend API Layer
- AI Processing Layer
- Storage and Session Management Layer

Each layer communicates through clearly defined interfaces described below.

1.2.1. Frontend–Backend Interface

The communication between the frontend and backend is implemented through RESTful HTTP APIs over HTTPS.

Protocol

- HTTPS (mandatory)
- JSON-based request/response format
- Multipart/form-data for image uploads

Core API Endpoints

POST /upload

- Purpose: Upload image for processing
- Input: Image file (JPEG, PNG)
- Output: Session ID and metadata

POST /process

- Purpose: Apply selected AI operation
- Input:
 - Session ID
 - Operation type (e.g., inpainting, background removal)
 - Optional parameters
- Output:
 - Processed image (Base64 or file response)
 - Status message

GET /status/{session_id}

- Purpose: Check processing state
- Output:
 - Processing status
 - Error message (if any)

Response Structure

All API responses follow a standardized structure:

```
{  
  "status": "success | error",  
  "message": "descriptive message",  
  "data": { ... }  
}
```

This uniform response format ensures predictable frontend behavior and simplifies error handling.

1.2.2. Backend–AI Processing Interface

The backend communicates with AI modules through an internal service layer. AI modules are abstracted using a processing interface.

AI Module Interface Contract

Each AI processing module must implement:

- `load_model()`
- `preprocess(input_image)`
- `infer(processed_input)`
- `postprocess(model_output)`

This abstraction ensures:

- Model independence
- Replaceability of AI models
- Clean separation of business logic and ML logic

Models are initialized at application startup to minimize runtime overhead.

1.2.3. Data Format Specifications

Image Input Requirements

- Supported formats: JPEG, PNG
- Maximum resolution: Defined system limit
- Maximum file size: Server-defined constraint

Images are internally converted to:

- RGB format
- Standard tensor representation for model compatibility

Session Data

Session data includes:

- Temporary image reference
- Selected operation
- Processing state

Session data is stored in volatile memory and is cleared after processing completion or timeout.

1.2.4. Error Handling Guidelines

All system components follow consistent error-handling rules:

Error Categories

- Validation Errors (invalid image format, size exceeded)
- Processing Errors (model inference failure)
- System Errors (internal server issues)

Error Response Structure

```
{
  "status": "error",
  "message": "Human-readable description",
  "error_code": "SYSTEM_DEFINED_CODE"
}
```

Errors must not expose internal stack traces or model implementation details to the client.

1.2.5. Security Interface Constraints

All interfaces must comply with security constraints:

- HTTPS enforced
- Input validation before processing
- File type and MIME verification
- Rate limiting applied at API gateway
- No persistent image storage without explicit user consent

Additionally, AI modules must not access external network resources during inference.

1.2.6. External Dependencies Interface

Invisio depends on external frameworks and libraries such as:

- Backend framework
- Deep learning framework
- Image processing libraries

These dependencies are abstracted through service layers to prevent tight coupling. Any future library replacement should not affect upper architectural layers.

1.2.7. Logging and Monitoring Interface

System logging follows structured logging principles:

- Timestamped log entries
- Operation ID tagging
- Error classification

Logs do not contain:

- Raw image data
- Personal user identifiers

Monitoring hooks may be integrated for performance tracking, but sensitive data is excluded from logging pipelines.

1.3. Engineering Standards

The development and documentation of Invisio follow recognized software engineering standards in order to ensure consistency, clarity, and maintainability throughout the project lifecycle. Although the project is developed in an academic context, professional engineering principles and structured documentation practices are adopted.

1.3.1. IEEE Documentation Principles

The overall documentation structure of Invisio is inspired by IEEE software engineering standards, particularly those related to Software Design Description (SDD) and Software Requirements Specification (SRS).

The following principles are applied:

- Hierarchical section numbering for traceability.
- Explicit documentation of architectural decisions and trade-offs.
- Formal definition of interfaces before implementation details.
- Consistent terminology across all documents.

The LLD refines these components into detailed module-level descriptions and interface specifications. This structured approach ensures alignment between requirements, architectural design, and implementation.

1.3.2. UML Modeling Standards

Unified Modeling Language (UML) is used as the primary modeling standard to represent the system structure and behavior of Invisio.

Standard UML 2.x notation is followed for diagram construction. The following diagram types are utilized:

Use Case Diagrams

Use case diagrams describe system functionality from the user's perspective. They define:

- System boundaries
- External actors
- Core user interactions (e.g., image upload, AI processing, result retrieval)

These diagrams help ensure that functional requirements are properly captured and reflected in the system design.

Class Diagrams

Class diagrams represent the static structure of the system. They model:

- Major classes and modules
- Relationships between components
- Abstraction layers
- Dependencies between services

These diagrams support modular design and reinforce separation of concerns.

Sequence Diagrams

Sequence diagrams illustrate runtime behavior and interaction flow between system components. They describe:

- Request lifecycle
- Order of method calls
- Component collaboration during image processing
- Alternative flows such as error handling

This behavioral modeling ensures that the dynamic system behavior aligns with the architectural constraints defined in the HLD.

Component Diagrams

Component diagrams represent high-level structural decomposition, including:

- Frontend layer
- Backend API layer
- AI processing layer
- Session management mechanisms

These diagrams validate architectural modularity and low coupling between subsystems.

1.3.3. Design and Architectural Principles

Invisio follows widely accepted engineering principles, which are reflected in both UML diagrams and implementation decisions:

- Separation of Concerns (SoC): Each layer has a clearly defined responsibility.
- Modularity: AI features are encapsulated in independent processing modules.
- Low Coupling and High Cohesion: Components interact through defined interfaces.
- Stateless Processing: Each request is handled independently to improve scalability and maintain architectural clarity.

These principles guide both modeling and implementation.

1.3.4. Coding Standards and Conventions

The implementation follows general coding best practices:

- Consistent naming conventions for classes and methods.
- Layered project organization.
- Clear separation between routing, business logic, and AI computation.
- Standardized error handling structure.
- Input validation before processing.

Configuration parameters are externalized to avoid hard-coded values and to improve maintainability.

1.3.5. Version Control and Configuration Management

The project uses version control to manage source code and track changes systematically.

- Git-based version control is adopted.
- Changes are tracked through structured commits.
- Dependency versions are controlled to ensure reproducibility.

This ensures stability and supports collaborative development practices.

1.3.6. Ethical and Data Handling Standards

Since Invisio processes user-generated images, ethical engineering considerations are applied:

- No permanent storage of images without explicit need.
- No hidden data collection.
- Clear processing flow transparency.
- Input validation to prevent misuse.

These practices align with privacy-by-design principles and responsible AI system development.

1.4. Definitions, Acronyms, and Abbreviations

This section defines the technical terms, acronyms, and abbreviations used throughout the Invisio design documents. The purpose is to ensure clarity, consistency, and shared understanding of terminology.

1.4.1. Definitions

Term	Definition
Invisio	An AI-powered web-based image editing system designed to perform Automated image enhancement and manipulation tasks while following privacy-by-design principles.
AI Processing Layer	The system layer responsible for executing deep learning models for image manipulation tasks.
Inference	The execution of a trained machine learning model on input data to generate output results.
Preprocessing	The transformation of raw input images into a format suitable for model inference (e.g., resizing, normalization).
Postprocessing	The conversion of model outputs into a user-consumable format such as an image file.
Session	A temporary processing context that maintains image and operation state during a user interaction.
Stateless Architecture	An architectural approach where each request is handled independently without persistent shared state.

Modular Architecture	A design approach in which system functionality is divided into independent components communicating via defined interfaces.
Processing Module	A software component that encapsulates a specific AI-based image manipulation task.
Validation	The process of verifying that input data meets defined format and security constraints before processing.
Latency	The time required to complete an image processing request.
Throughput	The number of processing requests the system can handle within a given time period.
Session Context	A temporary runtime environment that holds image data and processing state during a single request lifecycle.
Ephemeral Data	Data that exists only in volatile memory and is deleted immediately after processing.

1.4.2. Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface

HLD	High-Level Design
LLD	Low-Level Design
UML	Unified Modeling Language
IEEE	Institute of Electrical and Electronics Engineers
SRS	Software Requirements Specification
SDD	Software Design Description
SoC	Separation of Concerns
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
GPU	Graphics Processing Unit
CPU	Central Processing Unit
JSON	JavaScript Object Notation
NSFW	Not Safe For Work
CNN	Convolutional Neural Network
GAN	Generative Adversarial Network
MVC	Model-View-Controller

2. Packages

Invisio is designed as a web-based AI-powered image processing system built on a client–server

architecture. The system follows the Model–View–Controller (MVC) architectural pattern in order to achieve separation of concerns, modularity, and maintainability.

The server-side implementation is structured according to MVC principles, where:

- Model represents data structures and AI processing logic
- Controller handles request routing and system coordination
- View represents the user interface layer

The following subsections describe the system components from the server perspective.

2.1 Server

2.1.1 Model

Figure 1: Model Package Diagram (see Appendix-A)

The Model package contains the core business logic and data representations of the Invisio system. It is responsible for storing image data, managing editing requests, handling AI-based processing models, and maintaining session and community-related information. The model layer is independent from the user interface and controller logic and focuses on representing system entities and their relationships.

User: This class represents a user of the system. It stores basic authentication state and session information. A user can upload images, initiate editing operations, and interact with community features. The User class acts as the entry point for personalized image processing operations.

Session: This class represents an active session of a user. It stores session identifier and expiration information. Session is responsible for ensuring security by determining whether a user session is still valid or has expired. Session control supports the system's security vs. usability trade-off.

Image: This class represents an uploaded image entity in the system. It stores metadata such as format, size, and additional information along with raw image data. The Image class is used as the primary input for AI-based editing operations.

EditingRequest: This class encapsulates an image editing operation requested by the user. It includes the editing type and parameter map required by the selected AI model. EditingRequest enables flexibility by allowing different editing operations to be processed through a unified structure.

Mask: This class represents segmentation masks used during region-based editing operations such as inpainting and refinement. It stores mask pixel data and confidence information generated by AI models.

ResultImage: This class represents the output of an AI processing operation. It stores processed

image data and provides export functionality in different formats. Each editing operation generates a ResultImage instance.

CommunityImage: This class represents images shared within the community. It extends the functionality of ResultImage by storing additional information such as count. CommunityImage enables social interaction features within the system.

RefinementTool: This class provides manual refinement functionality after AI-based processing. It allows users to modify masks using brush operations. RefinementTool enhances precision by allowing user-controlled adjustments.

AIModel (Abstract): This abstract class defines the common interface for all AI-based image processing models in the system. It declares the process method which takes an image and editing request as input and produces a ResultImage. The system follows the Strategy design pattern through this abstraction.

SegmentationModel: This class extends AIModel and performs image segmentation operations. It generates masks identifying object boundaries or regions of interest.

InpaintingModel: This class extends AIModel and performs content-aware image filling operations based on provided masks.

EnhancementModel: This class extends AIModel and applies quality improvement operations such as sharpening and noise reduction.

StyleTransferModel: This class extends AIModel and applies artistic style transformations to images.

BeautificationModel: This class extends AIModel and performs portrait enhancement and facial improvement operations.

ColorizationModel: This class extends AIModel and performs automatic colorization of grayscale images.

NSFWDetectionModel: This class extends AIModel and detects inappropriate content in images to ensure platform safety.

AIModelFactory: This class is responsible for creating appropriate AIModel instances based on the requested editing type. It encapsulates object creation logic and supports extensibility of the system.

2.1.2 Controller

Figure 2: Controller Package Diagram (see Appendix-B)

The Controller package is responsible for managing user requests, coordinating model operations, and ensuring secure communication between the View and Model layers. Controllers handle input validation, session management, AI processing orchestration, preview generation, and community interactions. The system follows the MVC architecture where controllers act as intermediaries between user interface components and the core business logic implemented in the model layer.

ImageController: This controller is responsible for handling image upload and validation operations. It receives image files from the client, validates their format and integrity, and forwards valid images to the model layer for storage and further processing.

EditingController: This controller acts as a facade for AI-based editing operations. It receives editing requests from the client and coordinates the creation of appropriate AIModel instances through the factory mechanism. It encapsulates the complexity of AI processing and provides a unified interface for different editing types.

RefinementController: This controller is responsible for handling mask refinement operations. After an AI-generated mask is produced, this controller enables user-driven adjustments through refinement tools. It communicates with the RefinementTool class in the model layer.

DownloadController: This controller manages exporting and downloading processed images. It retrieves the final ResultImage from the model layer and prepares it in the requested format for the user.

SessionController: This controller is responsible for creating and managing user sessions. It handles session creation, expiration, and lifecycle management. SessionController supports security requirements by ensuring that only valid sessions can access editing operations.

SecurityController: This controller acts as an interceptor in the system. It validates authentication tokens, performs request validation, and applies rate limiting mechanisms. SecurityController ensures that malicious or unauthorized operations are prevented before they reach the business logic.

2.2 Client

2.2.1 View

Figure 3: View Package Diagram (see Appendix-C)

The View package contains user interface components of the Invisio system. It is responsible for rendering visual elements, collecting user input, and displaying processing results. The View layer does not contain business logic; instead, it communicates with the Controller layer to initiate operations and retrieve processed data.

The system follows the MVC architecture where the View layer focuses solely on presentation and user interaction.

MainLayout: This class represents the main layout of the application. It acts as the central container of the user interface and manages routing between different views. MainLayout is responsible for rendering the appropriate view component based on user navigation.

UploadView: This view is responsible for allowing users to upload images to the system. It provides the interface for selecting files and initiating the upload process through the controller layer.

EditingOptionsView: This view allows users to select different editing operations such as segmentation, inpainting, enhancement, or style transfer. It collects editing parameters and forwards the request to the controller layer.

RefinementView: This view enables users to manually refine AI-generated masks. It provides brush-based interaction and adjustment tools for improving editing precision.

PreviewView: This view displays temporary editing results before final confirmation. It allows users to inspect modifications and decide whether to proceed with download or further refinement.

DownloadView: This view allows users to export and download processed images. It provides format selection and triggers the download operation via the controller layer.

CommunityView: This view enables users to browse and share community images. It displays shared content and supports social interaction features.

ErrorView: This view is responsible for displaying error messages related to validation, authentication, or processing failures. It ensures that system errors are presented to users in a structured and understandable manner.

3. Class Interfaces

3.1 Server

3.1.1 Model

Class User

Description:

Represents a system user. Stores authentication state, session association, and community-related information. Acts as the entry point for personalized image processing workflows.

Attributes

Visibility	Type	Name	Description
private	String	userId	Unique identifier of the user

private	boolean	authenticated	Indicates authentication state
private	Session	activeSession	Associated active session
private	List<Image>	uploadedImages	Images uploaded by the user
private	List<CommunityImage>	sharedImages	Images shared to community

Methods

Visibility	Return Type	Method Signature	Description
public	void	authenticate(String token)	Authenticates user session
public	void	logout()	Terminates authentication
public	void	uploadImage(Image image)	Registers uploaded image
public	void	shareImage(CommunityImage image)	Publishes image to community

Class Session

Description:

Represents an active user session. Maintains expiration control and session validity to support security requirements.

Attributes

Visibility	Type	Name	Description
private	String	sessionId	Unique session identifier
private	DateTime	createdAt	Session creation timestamp
private	DateTime	expiresAt	Expiration timestamp
private	boolean	valid	Session validity flag

Methods

Visibility	Return Type	Method Signature	Description
public	boolean	isValid()	Checks session validity
public	void	invalidate()	Terminates session
public	void	refresh()	Extends expiration time

Class Image

Description:

Represents an uploaded image entity. Stores metadata and raw image data used for AI processing.

Attributes

Visibility	Type	Name	Description
private	String	imageId	Unique identifier
private	byte[]	rawData	Raw pixel data
private	String	format	Image file format (JPG, PNG)
private	long	size	File size in bytes
private	int	width	Image width
private	int	height	Image height
private	Map<String, Object>	metadata	Additional image metadata

Methods

Visibility	Return Type	Method Signature	Description
------------	-------------	------------------	-------------

public	boolean	validateFormat()	Validates image format
public	boolean	validateSize(long maxSize)	Validates file size
public	void	normalize()	Applies preprocessing normalization

Class EditingRequest

Description:

Encapsulates a user-initiated editing operation, including editing type and model parameters.

Attributes

Visibility	Type	Name	Description
private	String	requestId	Unique request identifier
private	EditingType	editingType	Requested editing operation
private	Map<String, Object>	parameters	Operation-specific parameters
private	DateTime	requestedAt	Request timestamp

Methods

Visibility	Return Type	Method Signature	Description
public	Object	getParameter(String key)	Retrieves parameter value
public	void	validate()	Validates request consistency

Class Mask

Description:

Represents segmentation mask data used in region-based editing operations.

Attributes

Visibility	Type	Name	Description
private	byte[][]	maskData	Pixel-level mask values
private	float	confidenceScore	AI confidence metric
private	int	width	Mask width
private	int	height	Mask height

Methods

Visibility	Return Type	Method Signature	Description
public	void	refine(byte[][] brushInput)	Applies refinement operations
public	boolean	validateDimensions(Image image)	Ensures mask-image compatibility

Class ResultImage

Description:

Represents the output of an AI processing operation. Provides export functionality.

Attributes

Visibility	Type	Name	Description
private	String	resultId	Unique result identifier
private	byte[]	processedData	Processed image data
private	String	format	Output format
private	DateTime	generatedAt	Generation timestamp

Methods

Visibility	Return Type	Method Signature	Description
public	File	export(ExportFormat format)	Converts image to specified format
public	byte[]	getData()	Returns processed image bytes

Class CommunityImage (extends ResultImage)

Description:

Represents images shared within the community.

Attributes

Visibility	Type	Name	Description
private	int	likeCount	Number of likes
private	int	viewCount	Number of views
private	String	ownerId	Owner user identifier
private	DateTime	sharedAt	Share timestamp

Methods

Visibility	Return Type	Method Signature	Description
public	void	incrementLikes()	Increases like count
public	void	incrementViews()	Increases view count

Class RefinementTool

Description:

Provides manual mask adjustment functionality.

Attributes

Visibility	Type	Name	Description
private	int	brushSize	Brush radius
private	float	brushStrength	Brush intensity

Methods

Visibility	Return Type	Method Signature	Description
public	Mask	applyRefinement(Mask mask, byte[][] brushInput)	Applies refinement to mask

Class AIModel (Abstract)

Description:

AIModel defines the common contract for all AI-based image processing strategies in the system. It implements the Strategy design pattern and enforces a unified processing interface for extensibility and maintainability.

Attributes

Visibility	Type	Name	Description
protected	String	modelName	Logical name of the model
protected	String	version	Model version identifier
protected	boolean	loaded	Indicates whether model weights are loaded
protected	int	inputWidth	Expected input width
protected	int	inputHeight	Expected input height
protected	String	device	Execution device (CPU / GPU)

Methods

Visibility	Return Type	Method Signature	Description
public	abstract ResultImage	process(Image image, EditingRequest request)	Executes model inference
public	void	loadModel()	Loads model weights into memory
public	void	unloadModel()	Releases model resources
public	boolean	isLoading()	Checks model state
protected	Image	preprocess(Image image)	Applies resizing, normalization
protected	ResultImage	postprocess(byte[] rawOutput)	Converts raw output to ResultImage

Class SegmentationModel (extends AIModel)

Description:

Performs semantic segmentation to generate region masks identifying objects or regions of interest.

Attributes

Visibility	Type	Name	Description
private	float	confidenceThreshold	Minimum mask confidence
private	boolean	multiClass	Indicates multi-class segmentation

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	process(Image image, EditingRequest request)	Generates segmentation result
public	Mask	generateMask(Image image)	Produces segmentation mask
private	Mask	filterLowConfidence(Mask mask)	Removes weak detections

Class InpaintingModel (extends AIModel)

Description:

Performs content-aware region filling using provided masks.

Attributes

Visibility	Type	Name	Description
private	boolean	edgeAware	Enables edge-preserving filling

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	process(Image image, EditingRequest request)	Performs inpainting
public	ResultImage	fillRegion(Image image, Mask mask)	Fills masked area

Class EnhancementModel (extends AIModel)

Description:

Applies quality improvement operations such as denoising, sharpening, and contrast optimization.

Attributes

Visibility	Type	Name	Description
private	float	sharpnessFactor	Controls sharpening intensity
private	float	noiseReductionLevel	Denoising strength

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	process(Image image, EditingRequest request)	Enhances image quality
private	Image	applySharpening(Image image)	Applies sharpening
private	Image	applyDenoising(Image image)	Applies denoising

Class StyleTransferModel (extends AIModel)

Description:

Applies artistic style transformations using neural style transfer techniques.

Attributes

Visibility	Type	Name	Description
private	String	styleId	Selected artistic style
private	float	styleStrength	Controls blending ratio

Methods

Visibility	Return Type	Method Signature	Description
------------	-------------	------------------	-------------

public	ResultImage	process(Image image, EditingRequest request)	Applies style transfer
public	ResultImage	applyStyle(Image image, String styleId)	Performs style transformation

Class BeautificationModel (extends AIModel)

Description:

Performs portrait enhancement and facial improvements using AI-based facial feature detection.

Attributes

Visibility	Type	Name	Description
private	float	smoothingLevel	Skin smoothing intensity
private	float	eyeEnhancementLevel	Eye enhancement strength

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	process(Image image, EditingRequest request)	Enhances facial features
private	Image	smoothSkin(Image image)	Applies smoothing
private	Image	enhanceEyes(Image image)	Enhances eye regions

Class ColorizationModel (extends AIModel)

Description:

Performs automatic colorization of grayscale images.

Attributes

Visibility	Type	Name	Description
------------	------	------	-------------

private	boolean	temperatureCorrection	Enables tone adjustment
---------	---------	-----------------------	-------------------------

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	process(Image image, EditingRequest request)	Colorizes grayscale image
private	Image	applyColorMapping(Image image)	Maps predicted color channels

Class NSFWDetectionModel (extends AIModel)

Description:

Detects inappropriate or unsafe content before processing to ensure platform compliance and safety.

Attributes

Visibility	Type	Name	Description
private	float	detectionThreshold	Probability threshold
private	boolean	autoBlock	Enables automatic blocking

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	process(Image image, EditingRequest request)	Validates image safety
public	boolean	detect(Image image)	Returns detection result

Class AIModelFactory

Description:

Responsible for creating and managing AIModel instances. Encapsulates object creation logic and supports Open/Closed Principle compliance.

Attributes

Visibility	Type	Name	Description
private	Map<EditingType, AIModel>	modelRegistry	Registered model mappings

Methods

Visibility	Return Type	Method Signature	Description
public	AIModel	createModel(EditingType type)	Returns appropriate model
public	void	registerModel(EditingType type, AIModel model)	Adds new model
public	boolean	supports(EditingType type)	Checks model availability

3.1.2 Controller Package – Class Interfaces

Class ImageController**Description:**

Handles image upload, validation, and temporary storage operations. Ensures that uploaded files comply with format, size, and integrity constraints before forwarding them to the processing pipeline.

Attributes

Visibility	Type	Name	Description
private	List<String>	allowedFormats	List of supported image file extensions
private	long	maxFileSize	Maximum allowed file size (in bytes)

private	SessionController	sessionController	Manages session-based image storage
private	SecurityController	securityController	Handles authentication and request validation

Methods

Visibility	Return Type	Method Signature	Description
public	Image	uploadImage(File imageFile)	Accepts and stores a validated image
private	boolean	validateFormat(String extension)	Checks whether file format is allowed
private	boolean	validateSize(long size)	Validates file size constraints
private	boolean	validateIntegrity(File imageFile)	Verifies file consistency and corruption
public	void	deleteTemporaryImage(String sessionId)	Removes image after session completion

Class EditingController <<Facade>>

Description:

Acts as a facade for AI-based editing operations. Coordinates model selection, request validation, and processing orchestration through the AIModelFactory.

Attributes

Visibility	Type	Name	Description
private	AIModelFactory	modelFactory	Factory responsible for AI model instantiation
private	SessionController	sessionController	Provides access to session images
private	SecurityController	securityController	Ensures request authorization

Methods

Visibility	Return Type	Method Signature	Description
public	ResultImage	applyEditing(String sessionId, EditingRequest request)	Applies selected editing operation
private	AIModel	selectModel(EditingType type)	Determines appropriate AI model
private	void	validateEditingRequest(EditingRequest request)	Ensures request parameters are valid

Class RefinementController

Description:

Manages mask refinement operations after AI-based segmentation. Enables user-driven correction and regenerates final output.

Attributes

Visibility	Type	Name	Description
private	RefinementTool	refinementTool	Provides mask correction functionality
private	SessionController	sessionController	Retrieves session-based mask/image
private	SecurityController	securityController	Ensures secure refinement operation

Methods

Visibility	Return Type	Method Signature	Description
public	Mask	refineMask(String sessionId, Mask originalMask, BrushInput input)	Applies user-driven mask adjustments
public	ResultImage	regenerateImage(String sessionId, Mask refinedMask)	Reprocesses image using updated mask

Class DownloadController

Description:

Responsible for exporting processed images into the desired format and initiating download operations.

Attributes

Visibility	Type	Name	Description
private	SessionController	sessionController	Retrieves processed image from session
private	SecurityController	securityController	Ensures secure download request

Methods

Visibility	Return Type	Method Signature	Description
public	File	prepareDownload(String sessionId, ExportFormat format)	Prepares processed image for export
private	File	convertFormat(ResultImage image, ExportFormat format)	Converts image into selected format

Class SessionController

Description:

Manages session lifecycle, temporary image storage, and cleanup operations to ensure privacy compliance.

Attributes

Visibility	Type	Name	Description
private	Map<String, Session>	activeSessions	Active session registry
private	long	sessionTimeout	Session expiration duration

Methods

Visibility	Return Type	Method Signature	Description
public	String	createSession()	Initializes a new session
public	boolean	validateSession(String sessionId)	Verifies session validity
public	void	expireSession(String sessionId)	Terminates session
public	void	storeImage(String sessionId, Image image)	Temporarily stores uploaded image
public	Image	getImage(String sessionId)	Retrieves image for processing
public	void	cleanupSession(String sessionId)	Deletes session data permanently

Class SecurityController <<Interceptor>>

Description:

Acts as a request interceptor to enforce authentication, authorization, and rate-limiting mechanisms before controller execution.

Attributes

Visibility	Type	Name	Description
private	TokenService	tokenService	Validates authentication tokens
private	RateLimiter	rateLimiter	Controls request frequency per user

Methods

Visibility	Return Type	Method Signature	Description
------------	-------------	------------------	-------------

public	boolean	validateToken(String token)	Validates authentication token
public	boolean	checkRateLimit(String userId)	Ensures request frequency compliance
public	void	interceptRequest(Request request)	Intercepts and validates request
public	void	rejectUnauthorized(Request request)	Blocks unauthorized access

3.2 Client

3.2.1 View

Class MainLayout <<Layout>>

Description:

Represents the root layout component of the application. It manages routing between view components and ensures consistent rendering of the application structure. MainLayout does not contain business logic and delegates user actions to the Controller layer.

Attributes

Visibility	Type	Name	Description
private	Map<String, View>	routes	Mapping of route paths to view components
private	View	currentView	Currently active view
private	String	currentRoute	Active route identifier

private	boolean	initialized	Indicates layout initialization state
---------	---------	-------------	---------------------------------------

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Renders current layout and active view
public	void	navigate(String route)	Changes active route and view
private	void	initializeRoutes()	Registers view components

Class UploadView

Description:

Provides the user interface for image selection and upload operations. Collects file input and forwards upload request to the Controller layer.

Attributes

Visibility	Type	Name	Description
private	File	selectedFile	Selected image file
private	boolean	uploadInProgress	Upload state indicator
private	String	errorMessage	Validation error message

private	ImageController	imageController	Controller dependency
---------	-----------------	-----------------	-----------------------

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Renders upload interface
public	void	selectFile(File file)	Handles file selection
public	void	submitUpload()	Initiates upload via controller
private	void	displayError(String message)	Displays validation error

Class EditingOptionsView

Description:

Allows users to select editing operations and configure parameters before processing.

Attributes

Visibility	Type	Name	Description
private	EditingType	selectedEditingType	Selected operation type
private	Map<String, Object>	parameters	Editing parameters

private	boolean	requestInProgress	Indicates processing state
private	EditingController	editingController	Controller dependency

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Displays editing options
public	void	selectEditingType(EditingType type)	Updates selection
public	void	updateParameter(String key, Object value)	Updates parameter map
public	void	submitEditingRequest()	Sends request to controller
private	void	displayLoading()	Shows processing indicator

Class RefinementView

Description:

Provides brush-based interaction for manual mask refinement after AI segmentation.

Attributes

Visibility	Type	Name	Description
------------	------	------	-------------

private	Mask	currentMask	Mask being refined
private	int	brushSize	Brush radius
private	boolean	refinementActive	Indicates refinement mode
private	RefinementController	refinementController	Controller dependency

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Renders refinement interface
public	void	applyBrush(int x, int y)	Applies brush action
public	void	submitRefinement()	Sends refined mask to controller
private	void	updatePreview()	Updates visual mask preview

Class PreviewView

Description:

Displays temporary AI processing results before final confirmation.

Attributes

Visibility	Type	Name	Description
private	ResultImage	previewImage	Temporary processed result
private	boolean	confirmationRequired	Indicates confirmation state
private	String	statusMessage	Displays processing status

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Renders preview screen
public	void	confirmResult()	Confirms and proceeds to download
public	void	requestRefinement()	Redirects to refinement view
private	void	displayResult(ResultImage image)	Updates preview content

Class DownloadView

Description:

Provides export options and initiates file download through the controller layer.

Attributes

Visibility	Type	Name	Description
private	ExportFormat	selectedFormat	Selected output format
private	boolean	downloadReady	Indicates availability
private	DownloadController	downloadController	Controller dependency

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Renders download options
public	void	selectFormat(ExportFormat format)	Updates selected format
public	void	initiateDownload()	Calls controller for export
private	void	displaySuccess()	Shows confirmation message

Class CommunityView

Description:

Displays shared community images and allows interaction such as viewing and liking.

Attributes

Visibility	Type	Name	Description
private	List<CommunityImage>	communityImages	Shared images list
private	String	selectedImageId	Selected image identifier
private	boolean	loading	Loading state indicator

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Displays community gallery
public	void	viewImage(String imageId)	Opens selected image
public	void	likeImage(String imageId)	Increments like count
private	void	refreshGallery()	Reloads community data

Class **ErrorView**

Description:

Displays structured system errors such as validation failures, authentication errors, or processing exceptions.

Attributes

Visibility	Type	Name	Description
private	String	errorCode	System error code
private	String	errorMessage	User-friendly message
private	String	errorDetails	Technical error details

Methods

Visibility	Return Type	Method Signature	Description
public	void	render()	Displays error screen
public	void	displayError(String code, String message)	Sets error content
public	void	redirectToHome()	Navigates to safe route

4. Glossary

- **AI (Artificial Intelligence)**

A computational paradigm that enables systems to perform tasks requiring human-like perception, reasoning, and decision-making. In Invisio, AI refers specifically to deep learning models used for image manipulation tasks such as segmentation, inpainting, enhancement, and style transfer.

- **Machine Learning (ML)**

A subfield of artificial intelligence that enables systems to learn patterns from data without explicit programming.

- **Deep Learning**

A machine learning approach based on multi-layer neural networks used for advanced image editing tasks.

- **Computer Vision**

A domain of artificial intelligence focused on enabling machines to interpret and analyze digital images.

- **AI Processing Layer**

The backend subsystem responsible for executing deep learning models. It encapsulates preprocessing, inference, and postprocessing operations while remaining isolated from presentation and routing logic under the MVC architecture.

- **CNN (Convolutional Neural Network)**

A deep neural network architecture optimized for visual feature extraction. CNNs form the backbone of segmentation, enhancement, and detection models within Invisio.

- **GAN (Generative Adversarial Network)**

A generative deep learning architecture composed of a generator and discriminator trained adversarially. GAN-based techniques are referenced in style transfer, beautification, and image synthesis modules.

- **Inference**

The execution phase of a trained machine learning model where input data (e.g., an image) is processed to produce an output result. In Invisio, inference occurs within the AIModel abstraction and must comply with standardized execution contracts.

- **Preprocessing**

The transformation of raw input images into a normalized tensor representation compatible with model input requirements. Typical operations include resizing, color normalization, and format conversion.

- **Postprocessing**

The conversion of raw model outputs (e.g., tensors, probability maps) into user-consumable artifacts such as image files or segmentation masks.

- **Segmentation**

The pixel-level classification of image regions into meaningful components (e.g., foreground vs background). Invisio's segmentation pipeline generates masks used for region-based editing tasks.

- **Image Segmentation**

The process of dividing an image into meaningful regions for object isolation and region-based editing.

- **Object Detection**

The identification and localization of objects within an image.

- **Inpainting**

A content-aware reconstruction technique that fills missing or removed image regions using contextual information. In Invisio, inpainting relies on mask-guided generative inference.

- **Neural Style Transfer**

A method that applies the visual style of one image to the structural content of another image.

- **Mask**

A binary or probabilistic pixel map indicating regions of interest in an image. Masks are generated by segmentation models and may be refined through the RefinementTool before final processing.

- **Enhancement**

AI-based improvement of image properties such as brightness, contrast, or clarity.

- **Beautification**

AI-based facial enhancement including skin smoothing and feature refinement.

- **Background Removal**

Automatic isolation of foreground elements from the background using segmentation techniques.

- **Background Replacement**

Replacing an existing image background with another image after foreground isolation.

- **Super-Resolution**

Increasing image resolution and detail using AI-based reconstruction techniques.

- **Colorization**

Automatically adding color to grayscale images using machine learning models.

- **Hybrid Editing Workflow**

An editing pipeline combining automated AI inference with optional human-guided refinement.

- **Human-in-the-Loop (HITL)**

A design paradigm where limited human interaction enhances automated AI outputs. In Invisio, this is implemented via structured mask refinement rather than full manual editing.

- **MVC (Model–View–Controller)**

An architectural pattern separating system concerns into:

- **Model:** Business logic and AI processing
- **View:** User interface components
- **Controller:** Request coordination and validation

Invisio follows MVC to ensure modularity, maintainability, and testability.

- **Stateless Architecture**

An architectural approach where each request is processed independently without persistent shared state. Invisio maintains session context in volatile memory only, avoiding long-term storage to preserve privacy.

- **Session**

A temporary processing context that exists during a single user interaction.

- **Session Context**

A temporary runtime environment maintaining image references, processing state, and request metadata during a single interaction lifecycle.

- **Session-Based Processing**

A model where user data exists only during an active session and is deleted afterward.

- **Ephemeral Data**

Data that exists only in volatile memory and is permanently deleted after session expiration or processing completion.

- **Volatile Memory**

Temporary memory used during processing that does not retain data after execution.

- **Privacy-by-Design**

An engineering principle that integrates data protection into system architecture from the outset. Invisio implements privacy-by-design by avoiding persistent image storage and long-term user profiling.

- **Data Minimization**

Processing only the minimum necessary data required for functionality.

- **GDPR (General Data Protection Regulation)**

European Union regulation governing personal data protection. Invisio's architecture minimizes data retention to align with GDPR principles.

- **KVKK (Kişisel Verilerin Korunması Kanunu)**

Türkiye's data protection regulation. Invisio's session-based processing model reduces compliance risks under KVKK.

- **TLS (Transport Layer Security)**

A cryptographic protocol ensuring secure data transmission over HTTPS.

- **HTTPS**

Secure communication protocol using TLS encryption over HTTP.

- **RESTful API**

An HTTP-based interface enabling communication between frontend and backend components.

- **Rate Limiting**

A security mechanism restricting the frequency of API requests per user or IP address to mitigate abuse and denial-of-service risks.

- **NSFW (Not Safe For Work)**

Content classification category identifying inappropriate or unsafe visual material. The NSFWDetectionModel filters such content before further processing.

- **Latency**

The total time required to process a user request from upload to response.

- **Throughput**

The number of processing requests the system can handle within a given time interval.

- **Strategy Pattern**

A behavioral design pattern enabling interchangeable algorithm implementations. Invisio applies the Strategy pattern via the AIModel abstraction.

- **Facade Pattern**

A structural design pattern providing a simplified interface to a complex subsystem. EditingController acts as a facade for heterogeneous AI models.

- **Interceptor Pattern**

A design pattern used to process or validate requests before reaching core logic. SecurityController applies the interceptor role for authentication and rate limiting.

- **Open/Closed Principle (OCP)**

A software design principle stating that software entities should be open for extension but closed for modification. AIModelFactory enables extensibility without altering existing controller logic.

- **Unified Modeling Language (UML)**

A standardized modeling language used to represent system architecture and behavior.

- **High-Level Design (HLD)**

Architectural description of system components and their interactions.

- **Low-Level Design (LLD)**

Detailed design describing classes, interfaces, and processing workflows.

- **Software Requirements Specification (SRS)**

A document defining functional and non-functional system requirements.

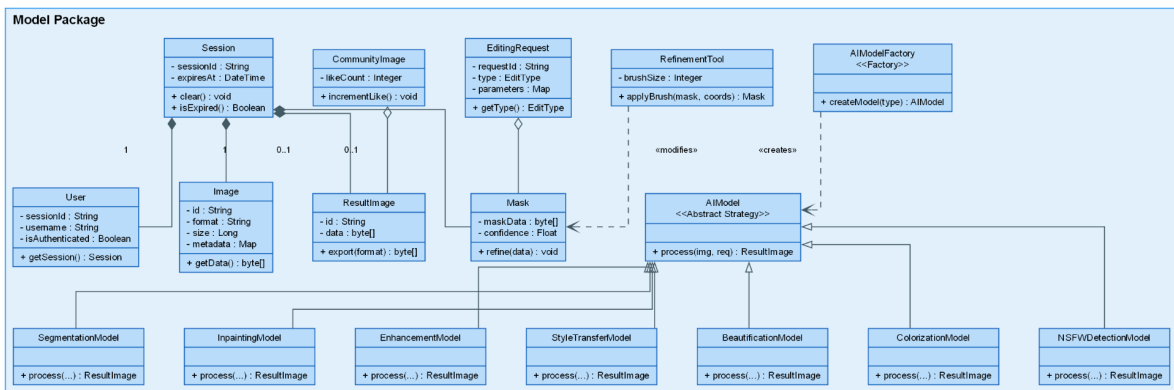
5. References

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., et al. (2014). *Generative adversarial nets*. NeurIPS.
- Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). *A neural algorithm of artistic style*. arXiv preprint.
- Jing, Y., Yang, Y., Feng, Z., Ye, J., Yu, Y., & Song, M. (2017). *Neural style transfer: A review*. IEEE Trans. on Visualization and Computer Graphics.
- Xu, Y. (2025). *Style transfer review: Traditional machine learning to deep learning*. Information, 16(2).
- Zhang, S., et al. (2025). *Applying deep learning for style transfer in digital art*. Scientific Reports.
- Diamant, N., Zadok, D., Baskin, C., Schwartz, E., & Bronstein, A. M. (2019). *Beholder-GAN: Generation and beautification of facial images with conditioning on their beauty level*. In *Proc. IEEE ICIP*.
- Gan, J., et al. (2024). *Applied research on face image beautification based on FIBGAN*. Electronics.
- Peng, T. (2024). *ISFB-GAN: Interpretable semantic face beautification*. Expert Systems with Applications.

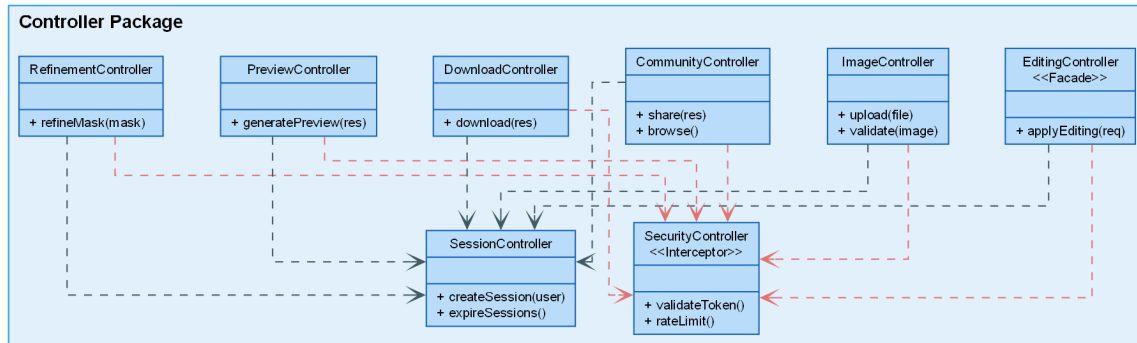
- Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation*. MICCAI.
- Ling, H., Kreis, K., Li, D., Kim, S. W., Torralba, A., & Fidler, S. (2021). *EditGAN: High-precision semantic image editing*. arXiv preprint.
- Anwar, S., Tahir, M., Li, C., Mian, A., Khan, F. S., & Muzaffar, A. W. (2020). *Image colorization: A survey and dataset*. arXiv preprint.
- [Author(s)] (2024). *[Survey on image colorization]*. Electronics.
- Gui, J., Sun, Z., Wen, Y., Tao, D., & Ye, J. (2020). *A review on Generative Adversarial Networks: Algorithms, theory, and applications*.
- Shamsolmoali, P., Zareapoor, M., Granger, E., Zhou, H., Wang, R., Celebi, M. E., & Yang, J. (2020). *Image synthesis with adversarial networks: A comprehensive survey and case studies*. arXiv preprint.
- Wu, X., Xu, K., & Hall, P. M. (2017). *A survey of image synthesis and editing with generative adversarial networks*. Tsinghua Sci. & Tech.
- Zhou, X. (2025). *Survey paper bridging the metrics gap in image style transfer*. [Journal].
- Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., & Hochreiter, S. (2017). *GANs trained by a two time-scale update rule converge to a local Nash equilibrium*. NeurIPS.
- [Author(s)] (2025). *A review on generative AI for text-to-image and image-to-image generation*. arXiv preprint.

6. Appendix

6.1 Appendix A - Figure 1: Model Package Diagram



6.2. Appendix B - Figure 2: Controller Package Diagram



6.3. Appendix C - Figure 3: View Package Diagram

